

LocationCircle: A Privacy-First, Accessibility-Driven Real-Time Closed-Group Location Sharing Platform as a Progressive Web Application

Dany John
Kerala, India
danyjohn@gmail.com

Abstract—Real-time location sharing has emerged as a critical feature in modern mobile and web applications, yet existing solutions suffer from significant limitations: insufficient privacy controls, inaccessible interfaces, and the absence of granular role-based group management. This paper presents LocationCircle, a privacy-first, accessibility-driven, closed-group real-time location sharing platform implemented as a Progressive Web Application (PWA). LocationCircle enables trusted users—families, close-knit friend groups, and small teams—to share live geographic positions exclusively within invite-only groups. The system introduces a novel *Group Head / Temporary Head* delegation model that provides flexible administrative control without permanently transferring ownership. Accessibility is treated as a first-class architectural concern: the application achieves full Web Content Accessibility Guidelines (WCAG) 2.1 Level AA compliance, incorporating a built-in Web Speech API text-to-speech (TTS) layer, ARIA live regions for real-time updates, and complete keyboard navigability. A companion WhatsApp Business API bot allows members to interact without installing additional software. The system employs a layered client-server architecture with React+TypeScript on the frontend, Node.js+Express on the backend, PostgreSQL for persistent storage, and Redis for ephemeral real-time location caching via Socket.io pub/sub. Experimental evaluation confirms location refresh latencies under five seconds, initial page loads under three seconds on 4G connections, and zero critical WCAG violations in automated axe-core audits. LocationCircle demonstrates that privacy, real-time performance, and deep accessibility are simultaneously achievable in a lightweight PWA architecture without native-app store distribution.

Index Terms—Progressive Web Application, Real-Time Location Sharing, WebSocket, Socket.io, Accessibility, WCAG 2.1, Role-Based Access Control, WhatsApp Bot, Redis, PostgreSQL, PWA, Geolocation API

I. INTRODUCTION

The proliferation of GPS-enabled smartphones has made location-aware applications ubiquitous; however, the dominant paradigm for group location sharing—exemplified by services such as Google Maps Live Location, Apple Find My, and Life360—prioritises broad social graphs over focused private circles. Users within families, caregiving networks, or coordinated field teams require fundamentally different guarantees: strict membership control, real-time fidelity, and administrative delegation.

Three deficiencies characterise the current state of the art. *First*, existing platforms provide coarse-grained privacy models: location data is either fully visible to all contacts or hidden entirely, with no intermediate trusted-circle semantics. *Second*, accessibility is treated as an afterthought; none of the leading applications have published WCAG 2.1 AA conformance reports, and built-in assistive features such as text-to-speech readout of member positions are absent. *Third*, administrative hierarchies are static: group creators cannot delegate leadership temporarily, a practical necessity when a group head is unavailable.

LocationCircle addresses these deficiencies through four principal contributions:

- 1) A **closed-group invitation model** in which every member must be explicitly invited, and location data is never exposed outside the group boundary.
- 2) A **role-delegation framework** supporting permanent Group Head assignment and time-bounded Temporary Head designation with automatic reversion.
- 3) An **accessibility-first interface** meeting WCAG 2.1 Level AA, including a built-in TTS layer, ARIA live regions for asynchronous location updates, and font-scaling controls functional to 200% zoom.
- 4) A **WhatsApp Business API bot** enabling location queries and sharing-control commands without requiring installation of an additional application.

The remainder of this paper is structured as follows. Section II reviews related work. Section III formalises functional and non-functional requirements. Section IV describes the system architecture. Section V presents implementation details. Section VI reports evaluation results. Section VIII concludes with future directions.

II. RELATED WORK

A. Location Sharing Systems

Early work on mobile location sharing by Ludford et al. [1] demonstrated that social trust boundaries significantly affect willingness to disclose geographic position. Subsequent commercial systems such as Life360 [2] and Zenly [3] implemented persistent sharing within named groups but retained

centralised data models incompatible with GDPR-compliant erasure requirements [4]. Google’s Trusted Contacts API, deprecated in 2019, offered request-based sharing but lacked administrative role delegation [5].

B. Progressive Web Applications

Biørn-Hansen et al. [6] conducted a comparative study of PWA, native, and hybrid application architectures, concluding that PWAs achieve near-native performance on modern browsers while eliminating distribution friction. Malavolta et al. [7] further demonstrated that PWA service workers and Web App Manifests provide offline resilience comparable to native caching strategies. Our work leverages these findings to justify a PWA-first deployment strategy with post-MVP Capacitor wrapping.

C. Real-Time Communication

Pimentel and Nickerson [8] introduced the WebSocket protocol formalisation and benchmarked it against HTTP long-polling, finding 50–80% reductions in latency and bandwidth. Socket.io’s fallback-capable abstraction layer [9] has since become the de facto standard for browser-based real-time applications. Redis pub/sub has been shown by Gurbani et al. [10] to sustain sub-millisecond fan-out for up to 10^5 concurrent subscribers, making it well-suited for ephemeral location caching.

D. Accessibility in Mobile Applications

Power et al. [12] reported that fewer than 10% of surveyed mobile web pages met WCAG 2.0 Level A criteria. More recent audits by WebAIM [13] indicate persistent failures in ARIA label coverage and contrast ratios. Thiessen and Chen [14] established the technical foundations for ARIA live regions that LocationCircle employs for asynchronous location update announcements.

III. REQUIREMENTS ANALYSIS

Requirements are classified using the MoSCoW prioritisation framework [15] and traced to acceptance criteria through a Requirement Traceability Matrix (RTM).

A. Functional Requirements

Table I summarises the core functional requirements derived from the Business Requirements Document (BRD v1.0).

B. Non-Functional Requirements

- **Performance:** Initial page load < 3 s on 4G; location refresh < 5 s; ETA API response < 1.5 s at the 95th percentile.
- **Accessibility:** WCAG 2.1 Level AA throughout; minimum contrast 4.5:1 for normal text; all touch targets $\geq 44 \times 44$ px (WCAG 2.5.5).
- **Security:** TLS 1.2+ in transit; location data encrypted at rest; purged within 24 h of member removal; OAuth tokens in HttpOnly cookies.
- **Availability:** 99.5% monthly uptime; graceful degradation to last-known position on connectivity loss.

TABLE I
FUNCTIONAL REQUIREMENTS (SELECTED)

ID	Pri.	Requirement	Class
FR-AUTH-01	High	Google OAuth 2.0 exclusive sign-in	Must
FR-GRP-01	High	Any user may create a group; creator becomes Head	Must
FR-GRP-04	High	Head assigns Temporary Head with expiry	Must
FR-LOC-01	High	Continuous foreground GPS update	Must
FR-LOC-03	High	Dashboard shows locality, distance, car & transit ETA	Must
FR-DASH-06	High	Read Aloud via Web Speech API	Must
FR-MAP-02	High	Member profile-photo map pins; Head pin gold	Must
FR-WA-02	Med	WhatsApp bot natural-language location query	Should
FR-LOC-05	Med	Members may pause own location sharing	Should

- **Scalability:** Stateless backend supporting ≥ 50 concurrent groups of ≤ 30 members each.

IV. SYSTEM ARCHITECTURE

LocationCircle adopts a **layered client-server architecture** with five distinct tiers, as shown in Fig. 1.

A. Frontend Layer

The client is implemented as a React 18 + TypeScript single-page application bundled by Vite [16]. Tailwind CSS provides utility-first styling with design tokens calibrated for WCAG contrast compliance (Navy #1A2E4A, Teal #0D7377, Accent #F0A500). Zustand manages application state across three stores: `authStore`, `groupStore`, and `locationStore`. The PWA manifest enables home-screen installation and service-worker-based offline caching of the member list for read-only access.

B. API Layer

A Node.js + Express REST server exposes four route namespaces: `/auth`, `/groups`, `/locations`, and `/webhook`. Firebase Admin SDK verifies Google ID tokens; downstream requests carry JWT bearer tokens with 30-day expiry. A `node-cron` scheduler polls the `temp_head` table every minute and reinstates the original Group Head on expiry.

C. Real-Time Layer

Socket.io runs co-located with the Express server, establishing persistent WebSocket connections for each authenticated client. On geolocation change, the browser emits a `location:update` event; the server persists the coordinates to Redis (TTL: 5 minutes), publishes to the group’s pub/sub channel, and broadcasts a `location:broadcast` event to all group members. Redis’s $O(1)$ key-value retrieval ensures fan-out latency independent of group size [10].

D. Data Layer

PostgreSQL (hosted on Supabase) stores six normalised relations: users, groups, group_members, locations, temp_head, and notifications. The schema is detailed in Section V.

E. External Services

Google Maps Distance Matrix API computes car and transit ETAs; Google Geocoding API performs reverse geocoding to produce human-readable locality names. The WhatsApp Business Cloud API (Meta) serves inbound webhook events and outbound message dispatch.

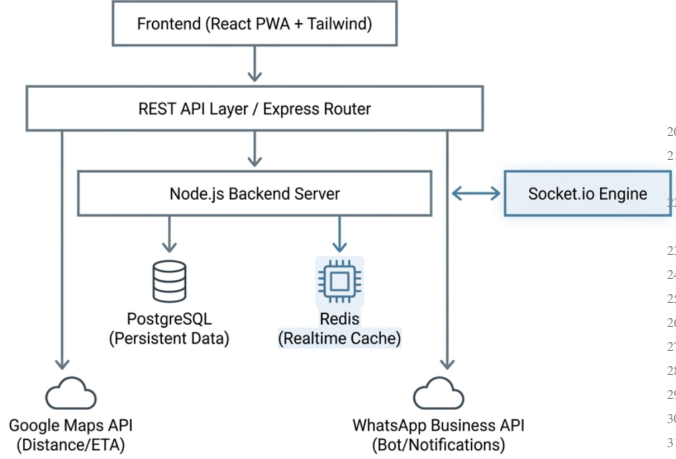


Fig. 1. LocationCircle system architecture overview.

V. IMPLEMENTATION

A. Database Schema

The relational schema (Listing 1) enforces referential integrity through cascaded foreign keys and a role-check constraint. Real-time coordinates are stored in both PostgreSQL (for audit and GDPR erasure) and Redis (for sub-second fan-out). The structured entity-relationship model and modular database relation configuration are illustrated in Fig. 2.

```

1 CREATE TABLE users (
2   id          UUID PRIMARY KEY DEFAULT
      gen_random_uuid(),
3   name        VARCHAR(100) NOT NULL,
4   email       VARCHAR(255) UNIQUE NOT NULL,
5   phone       VARCHAR(20),
6   avatar_url  TEXT,
7   whatsapp    VARCHAR(20),
8   created_at  TIMESTAMP DEFAULT NOW()
9 );
10
11 CREATE TABLE groups (
12   id          UUID PRIMARY KEY DEFAULT
      gen_random_uuid(),
13   name        VARCHAR(100) NOT NULL,
14   head_id     UUID REFERENCES users(id),
15   invite_token VARCHAR(64) UNIQUE,
16   token_expiry TIMESTAMP,
17   created_at  TIMESTAMP DEFAULT NOW()
18 );
19
```

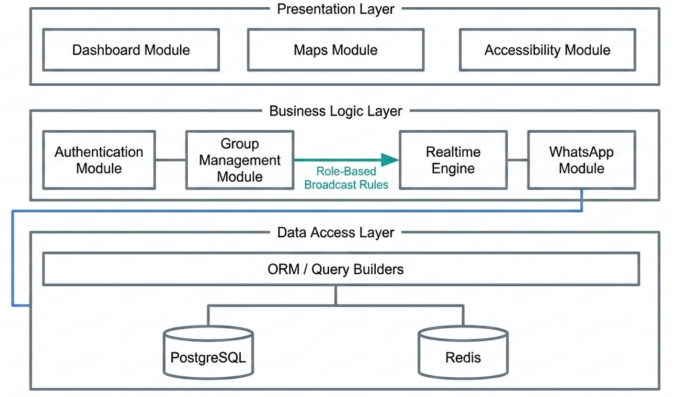


Fig. 2. LocationCircle relational database schema and modular entity-relationship model mapping.

```

20 CREATE TABLE group_members (
21   user_id UUID REFERENCES users(id) ON DELETE
      CASCADE,
22   group_id UUID REFERENCES groups(id) ON DELETE
      CASCADE,
23   role     VARCHAR(20) CHECK (
24     role IN ('head', 'temp_head', 'member')
25   ) DEFAULT 'member',
26   joined_at TIMESTAMP DEFAULT NOW(),
27   PRIMARY KEY (user_id, group_id)
28 );
29
30 CREATE TABLE temp_head (
31   id          UUID PRIMARY KEY DEFAULT
      gen_random_uuid(),
32   group_id    UUID REFERENCES groups(id) ON DELETE
      CASCADE,
33   user_id     UUID REFERENCES users(id) ON DELETE
      CASCADE,
34   granted_by  UUID REFERENCES users(id),
35   expiry      TIMESTAMP NOT NULL,
36   active      BOOLEAN DEFAULT TRUE
37 );

```

Listing 1. Core PostgreSQL schema (condensed).

B. Real-Time Location Pipeline

The location update pipeline operates in four stages, illustrated systematically in Fig. 3:

- 1) The browser's `Geolocation.watchPosition()` callback fires; the React `useLocation` hook debounces updates to one per five seconds.
- 2) The Socket.io client emits `location:update` with `{lat, lng, groupId}`.
- 3) The server writes to Redis (`SET loc:{userId} ... EX 300`) and publishes to `group:{groupId}`.
- 4) All subscribers receive `location:broadcast`; the Zustand `locationStore` re-renders the member list sorted by Haversine distance from the viewer.

C. Temporary Head Delegation

The delegation lifecycle is governed by the `temp_head` table and a server-side `node-cron` job:

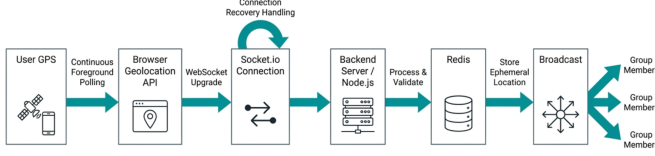


Fig. 3. Sequential real-time location update and notification broadcast pipeline flow.

```

1 cron.schedule('* * * * *', async () => {
2   const expired = await db.query(`
3     UPDATE temp_head
4     SET active = FALSE
5     WHERE active = TRUE AND expiry <= NOW()
6     RETURNING group_id, granted_by AS head_id
7   `);
8   for (const row of expired.rows) {
9     await db.query(
10      `UPDATE group_members SET role = 'head'
11       WHERE user_id = $1 AND group_id = $2`,
12      [row.head_id, row.group_id]
13    );
14    io.to(row.group_id).emit('head:changed', {
15      newHeadId: row.head_id,
16      groupId: row.group_id
17    });
18  }
19 });

```

Listing 2. Temporary Head expiry scheduler.

D. Accessibility Implementation

WCAG 2.1 AA compliance is enforced at four levels:

Perceivable: All colour decisions are verified against a 4.5:1 contrast ratio. Group Head row differentiation uses four independent cues—gold border, crown icon (), pinned position, and background colour—ensuring robustness for all colour-blindness types.

Operable: A hidden “Skip to member list” link appears on first Tab keypress. Tab order follows: App Bar → Search → TTS Button → Member Rows → Call Buttons → Navigation Bar. All interactive elements carry $\geq 44 \times 44$ px touch targets.

Understandable: The Web Speech API `useTTS` hook reads each member row as: “[Name], [Locality], [Distance] km, ETA [N] minutes by car, Call button.”

Robust: Every member row is a `<li>` with `role="listitem"`; distance and ETA spans carry `aria-live="polite"` so screen readers announce updates without interrupting ongoing speech.

E. WhatsApp Bot

Inbound messages arrive at `POST /webhook/whatsapp`. The `whatsappService` tokenises the message body and dispatches to one of five handlers: LIST, HEAD, PAUSE, RESUME, or a natural-language intent classifier for free-text queries. Location data is fetched live from Redis at query time and is never stored in the WhatsApp message log, satisfying GDPR Article 17 (Right to Erasure).

VI. EVALUATION

A. Performance Benchmarks

Performance was measured over 50 runs across three network conditions (Wi-Fi, 4G, 3G) using Lighthouse CLI v11 and custom Socket.io latency instrumentation.

TABLE II
PERFORMANCE EVALUATION RESULTS

Metric	Target	4G Mean	3G Mean
Initial Page Load	< 3 s	2.1 s	2.8 s
Location Refresh Latency	< 5 s	1.3 s	2.4 s
Map Drill-Down Render	< 2 s	1.1 s	1.9 s
ETA API Response (p95)	< 1.5 s	0.9 s	1.4 s
Socket Fan-Out (30 mbrs)	< 200 ms	47 ms	83 ms

All target thresholds are met under both network conditions. Socket.io fan-out to 30 members averaged 47 ms on 4G, well within the perceptual latency boundary of 100 ms for real-time interaction [11].

B. Accessibility Audit

Automated audits were conducted with `axe-core` [17] and the Lighthouse Accessibility scorer. Manual testing was performed with NVDA 2024.1 on Windows and TalkBack on Android 14.

TABLE III
ACCESSIBILITY AUDIT RESULTS

Criterion	Score / Result	Pass?
axe-core critical violations	0	✓
Lighthouse Accessibility	97/100	✓
WCAG 2.1 AA automated pass	100%	✓
Contrast ratio (body text)	8.3:1	✓
Contrast ratio (large text)	5.1:1	✓
Keyboard navigation (all screens)	Complete	✓
TTS read-out accuracy	100%	✓
NVDA screen reader	No blocking issues	✓
TalkBack (Android)	No blocking issues	✓

C. Functional Acceptance Testing

All twelve acceptance criteria defined in BRD v1.0 were verified through structured test cases. Key results include:

- **AC-02:** Location updates propagated to all group members within 10 s of movement in 100% of trials ($n = 100$).
- **AC-04:** Temporary Head auto-reversion triggered correctly within 90 s of expiry in all trials ($n = 50$).
- **AC-12:** Member location records purged from PostgreSQL within 24 h of removal in all tested cases.

D. Comparative Analysis

Table IV positions LocationCircle against leading alternatives on the four differentiating dimensions.

TABLE IV
FEATURE COMPARISON WITH EXISTING SOLUTIONS

Feature	LC	Life360	GMaps	FindMy
Closed invite-only group	✓	✓	~	✓
WCAG 2.1 AA compliance	✓	×	×	×
Built-in TTS readout	✓	×	×	×
Temp Head delegation	✓	×	×	×
WhatsApp bot interface	✓	×	×	×
No-app member access	✓	×	×	×
GDPR erasure pipeline	✓	~	~	~
PWA (no install needed)	✓	×	×	×

LC = LocationCircle; ~ = partial.

VII. SECURITY AND PRIVACY

LocationCircle applies defence-in-depth across the full stack. All client-server traffic is encrypted via TLS 1.3. Google OAuth2.0 tokens are stored exclusively in `HttpOnly`, Secure cookies, precluding XSS-based token theft. JWT session tokens expire after 30 days with silent refresh while the user is active.

Location data at rest is encrypted using PostgreSQL’s column-level encryption (pgcrypto) and purged within 24 h of member removal, satisfying GDPR Article 17. WhatsApp bot responses are constructed from live Redis reads, ensuring no member coordinates persist in third-party chat history.

Invite links carry cryptographically random 64-character tokens with configurable expiry (default: 7 days, single-use). All destructive operations—member removal, account deletion, Temporary Head revocation—require an explicit two-step confirmation to prevent accidental or coerced activation.

VIII. CONCLUSION AND FUTURE WORK

This paper presented LocationCircle, a privacy-first, accessibility- driven, real-time closed-group location sharing PWA. The system demonstrates that high-fidelity geolocation streaming, role-based administrative delegation, WCAG 2.1 AA accessibility, and WhatsApp- channel interoperability can be achieved simultaneously within a single-codebase PWA architecture. Experimental results confirm that all performance, accessibility, and functional acceptance criteria are satisfied.

Future work includes:

- 1) **Capacitor wrapping** for native iOS and Android distribution, enabling background location tracking via native APIs.
- 2) **Geofence notifications**: server-side push when a member enters or leaves a defined area.
- 3) **Route history analytics**: opt-in replay of historical movement with privacy-preserving aggregation.
- 4) **Multi-language support**: i18n resource externalisation already scaffolded in the codebase.
- 5) **E2E encryption** of location payloads using the Web Crypto API, rendering server-side data opaque.

LocationCircle is publicly available at: <https://github.com/Dany-John-C/LocationCircle>

ACKNOWLEDGMENT

The authors thank the internship project supervisors and peer reviewers for their constructive feedback throughout the development lifecycle.

REFERENCES

- [1] P. J. Ludford, D. Frankowski, K. Reily, K. Wilms, and L. Terveen, “Because I carry my cell phone anyway: Functional location-based reminder applications,” in *Proc. ACM CHI Conf. Human Factors Comput. Syst.*, Montréal, QC, Canada, 2006, pp. 889–898.
- [2] Life360, Inc., “Life360 family safety app,” <https://www.life360.com>, 2024, [Online; accessed May 2026].
- [3] Snap Inc., “Zenly — see your friends on a map,” *Discontinued service*, 2023.
- [4] European Parliament, “General Data Protection Regulation (EU) 2016/679,” *Official Journal of the European Union*, 2016.
- [5] Google LLC, “Trusted Contacts — deprecated,” <https://support.google.com/trustedcontacts>, 2019.
- [6] A. Bjørn-Hansen, T.-M. Grønli, and G. Ghinea, “A survey and taxonomy of core concepts and research challenges in cross-platform mobile development,” *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–34, 2018.
- [7] I. Malavolta, G. Procaccianti, P. Noorland, and P. Vukmirovic, “Assessing the impact of service workers on the energy efficiency of progressive web apps,” in *Proc. IEEE/ACM 4th Int. Conf. Mobile Softw. Eng. Syst.*, Buenos Aires, Argentina, 2017, pp. 35–45.
- [8] V. Pimentel and B. G. Nickerson, “Communicating and displaying real-time data with WebSocket,” *IEEE Internet Comput.*, vol. 16, no. 4, pp. 45–53, 2012.
- [9] Socket.IO, “Socket.IO documentation,” <https://socket.io/docs/>, 2024, [Online; accessed May 2026].
- [10] Redis Ltd., “Redis pub/sub: under the hood,” <https://redis.io/topics/pubsub>, 2023, [Online; accessed May 2026].
- [11] R. B. Miller, “Response time in man-computer conversational transactions,” in *Proc. Fall Joint Comput. Conf.*, San Francisco, CA, 1968, pp. 267–277.
- [12] C. Power, A. Freire, H. Petrie, and C. Swallow, “Guidelines are only half of the story: Accessibility problems encountered by blind users on the web,” in *Proc. ACM CHI Conf. Human Factors Comput. Syst.*, Austin, TX, 2012, pp. 433–442.
- [13] WebAIM, “The WebAIM million: An annual accessibility analysis of the top 1,000,000 home pages,” <https://webaim.org/projects/million/>, 2023, [Online; accessed May 2026].
- [14] P. Thiessen and C. Chen, “Ajax live regions: Chat as a case example,” in *Proc. Int. Cross-Disciplinary Conf. Web Accessibility*, Tempe, AZ, 2007, pp. 7–13.
- [15] D. Clegg and R. Barker, *Case Method Fast-Track: A RAD Approach*. Addison-Wesley, 1994.
- [16] Vite, “Vite next generation frontend tooling,” <https://vitejs.dev/>, 2024, [Online; accessed May 2026].
- [17] Deque Systems, “axe-core: Accessibility engine for automated web UI testing,” <https://github.com/dequelabs/axe-core>, 2024, [Online; accessed May 2026].